ARCHITECTING AGENTIC AI SOLUTIONS

PHYSICAL ARCHITECTURE

TAN LEE CHIANG

© 2011-25 National University of Singapore. All Rights Reserved.

Objectives

- Understand the different architecture styles, components and options to **integrate** components
- Understand the key architectural components
- Understand the different deployment models
- Ability to create a physical architecture for AI agentic solution

© 2011-24 National University of Singapore. All Rights Reserved.

Topics

- **Architectural styles and components**
- Deployment models
- Physical architectures for AI Agentic Solution

© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

Call and Return

- Each component **executes** only when it **gets the control** from another component and **returns** the control to that component when it **terminates execution**
- E.g., Open Systems Interconnection (OSI) layers, Operating System layers

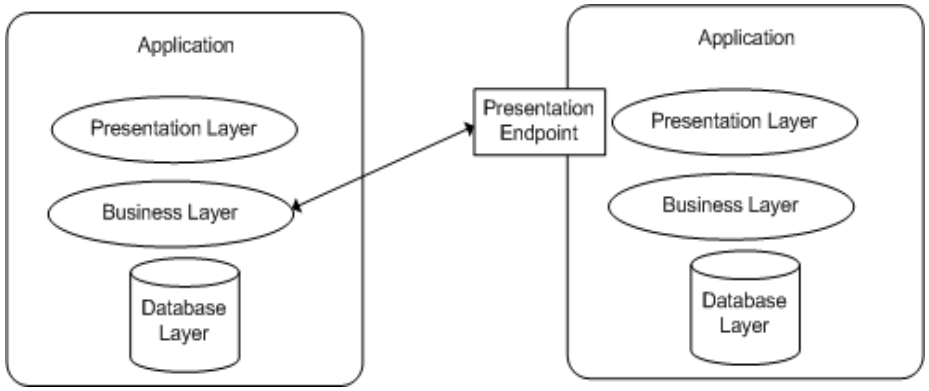
© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

Call and Return - Presentation Level

- Usage Scenarios
  - Extending **legacy** systems to the Web
  - Access to **public information** without compromising security
  - Absence of **source code**
  - Typically used for communication across **different technology domains**

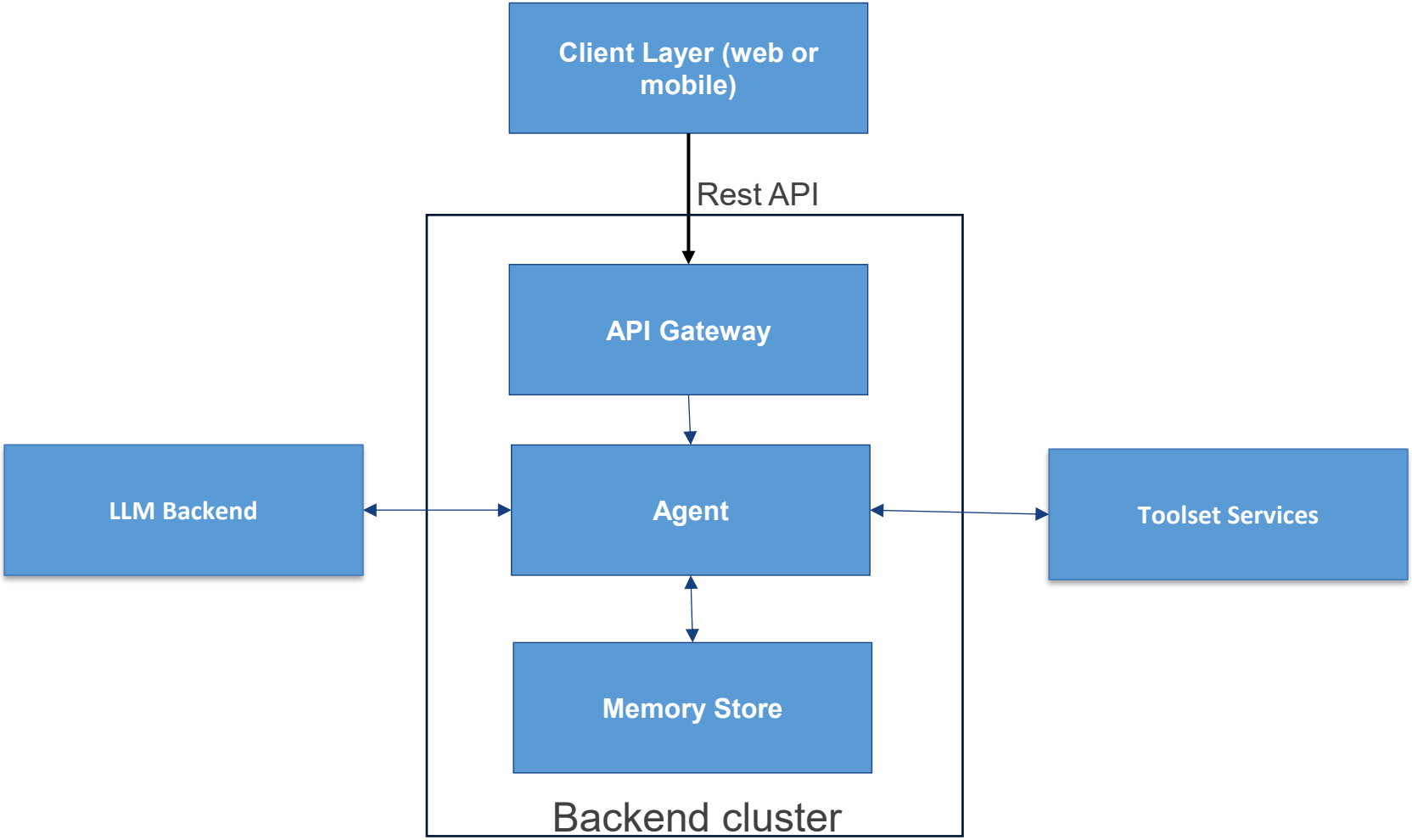
- Integration Technology
  - Accessing Web Page via URL
  - Screen scraping



© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

Call and Return – API Gateway



© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

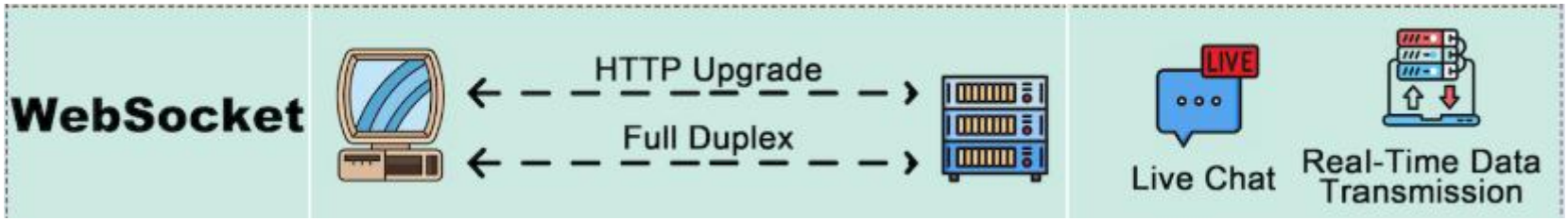
Call and Return – API Gateway in AI Agents

- Client Layer: Web/Mobile UI sending user requests
- API Gateway: Entry point, handles routing, auth, logging
- Agent: Controls agentic execution loop (Perceive → Reason → Act → Reflect)
- LLM Backend: Language model for reasoning and response (e.g ChatGPT)
- Memory Store: Vector DB (e.g., Pinecone) for long-term context
- Toolset Services: External tool handlers (e.g., email, search, planner)

© 2011-24 National University of Singapore. All Rights Reserved.

Full Duplex – WebSocket

- Unlike the traditional call and return which is a one way communication, WebSocket supports 2 way communication between 2 components
- Once the initial handshake (using HTTP), a WebSocket connection is created, and both components can send messages to each other
- As long the connection is not closed, the communication is open without the need to reconnect



Source: bytebytego.com

© 2011-24 National University of Singapore. All Rights Reserved.

WebSocket – Agentic AI Use Cases

Multi-agent Robotics: Drone fleets mapping terrain or swarm robots constructing structures. (WebSocket could be one of the way for the drones to communicate)

© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

WebSocket – Agentic AI Use Cases

Collaborative Software Agents: AI assistants co-editing documents or co-piloting software development.

Real-time Strategy AI: Agents working together to play games like StarCraft *(sorry that's the game I played when I was young)*

Negotiation & Decision-Making: AI agents engaging in live negotiations or auctions or selling

© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

WebSocket – Agentic AI Use Cases

Collaborative Software Agents: AI assistants co-editing documents or co-piloting software development.

Real-time Strategy AI: Agents working together to play games like StarCraft *(sorry that's the game I played when I was young)*

Negotiation & Decision-Making: AI agents engaging in live negotiations or auctions or selling

© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

Asynchronous Communication Style

Characteristic:

- Sender and receiver do not need to interact at the same time nor does it need to wait for reply
- Message can be stored and can be processed by the receiver when it is ready

© 2011-24 National University of Singapore. All Rights Reserved.

Asynchronous Communication Style

Simple examples in daily life:

- Email: You send an email now. The recipient reads it hours later
- Text messages: You send a message, the person replies when convenient

© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

Asynchronous Communication Style

Benefits:

- Doesn’t block the system and hence better performance
- Real time updates triggered the receiver to process without the need to have a job to periodically scan for new updates
- Message can be stored and can be processed by the receiver when it is ready

© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

Asynchronous Communication Style

Benefits:

- Components in the system are decoupled and hence it won't 'pull down' the whole system should one component is faulty
- Handling multiple concurrent connections

© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

Asynchronous Communication Style

Tools that are used

- Message Queues (RabbitMQ, Kafka): Decoupled messaging
- Event-driven frameworks: Node.js, asyncio in Python

© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

Asynchronous Communication Style

Event Driven Architecture (EDA)

- **Software design pattern** in which components (services, functions, or systems) communicate and respond to **events**—which are significant changes in state or occurrences—rather than through direct calls

Key Components

- **Event** - A record of an action or change of state
- **Event Producer** - The component that detects or triggers the event
- **Event Consumer** - The component that listens for and reacts to the event
- **Topic** – a logical channel of events
- **Event Channel / Broker** - Middleware that routes events from producers to consumers (e.g., Kafka, RabbitMQ, AWS EventBridge)

© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

Asynchronous Communication Style

Example: Develop AI Agents in a Rescue Simulation

© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

Asynchronous Communication Style

Example: Develop AI Agents in a Rescue Simulation Agents:

- **DroneAgent** - mapping, surveillance, and situational awareness
- **CommandAgent** - central coordinator and decision maker for the entire operation
- **RescueAgent** - Executes the physical rescue operation in the field

© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

Asynchronous Communication Style

Example: Develop AI Agents in a Rescue Simulation
Sequence of events:

- A **DroneAgent** surveyed and found a victim published ***"victim_found"*** event to the ***Event Bus*** (eg RabbitMq) that shared with **CommandAgent**

- **CommandAgent** been the subscriber will pick up the event to process and published ***"victim_rescue"*** event to ***Event Bus*** that shared with **RescueAgent**

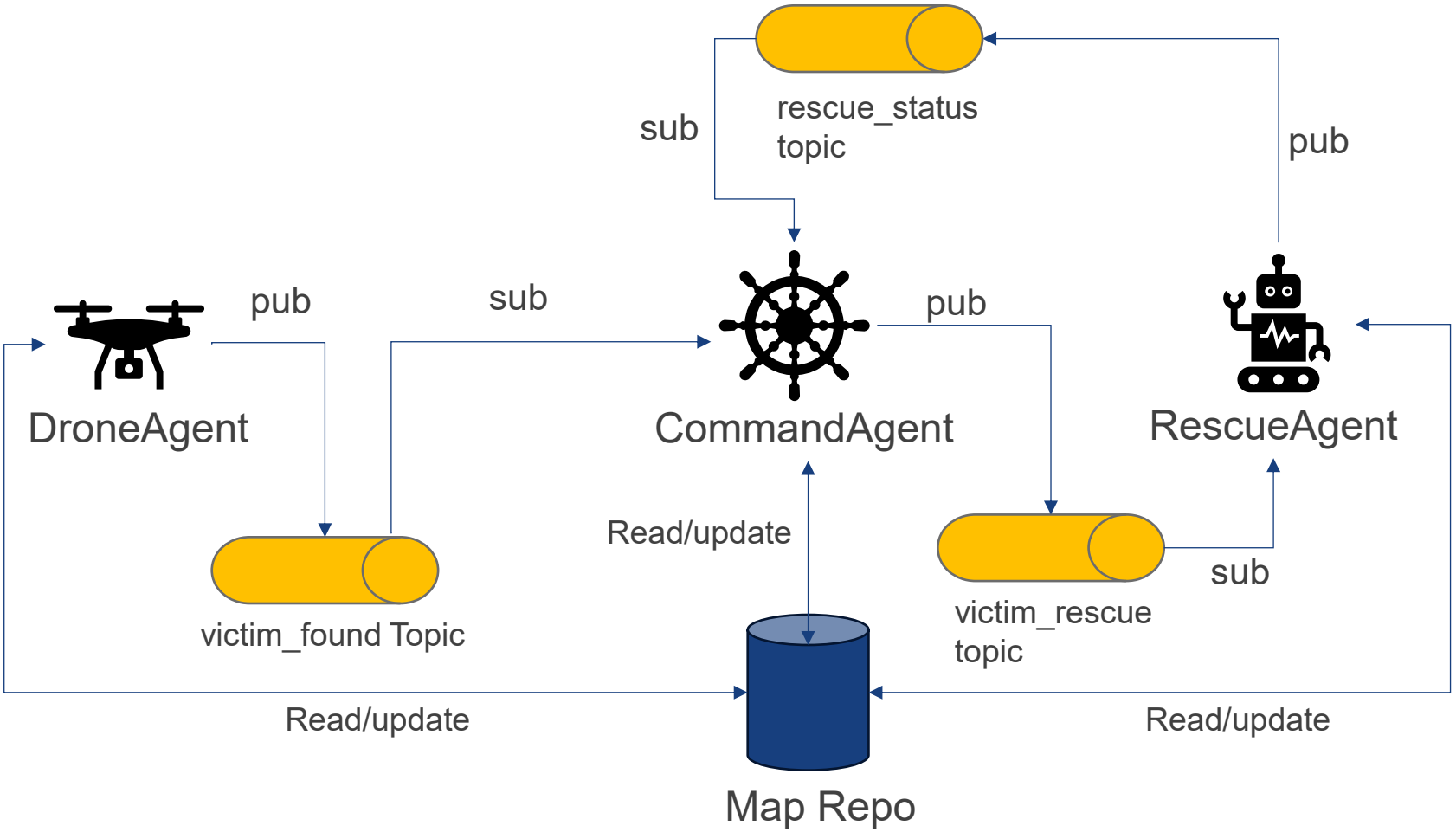
- **RescueAgent** been the subscriber will come to the victim's rescue and published "***rescue_complete***" event to the Event Bus that shared with **CommandAgent**

- **Command Agent** pick up the event update the map repo that the victim is rescued

© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

Asynchronous Communication Style



© 2011-24 National University of Singapore. All Rights Reserved.

Asynchronous Communication Style

Scalable

This solution could improve the following easily:

- Scale up and down the number of DroneAgent and RescueAgent
- A cache layer such as using RedisCache to replicate the Map Repo.

All these can be achieved easily because the components are very ***loosely coupled*** !

Resilient

Another benefit of EDA is should any component is not functioning, the messages in the topic will be stored and re-consumed by the subscribers when it is functioning again

© 2011-24 National University of Singapore. All Rights Reserved.

Architectural Component

Message Broker

Decouples producers and consumers

- Producers (senders) don't need to know about the consumers (receivers), and vice versa.

Message routing

- Directs messages to the appropriate destination(s) based on rules, topics, or queues.

Message transformation

- Converts messages from one format to another if needed.

Reliability & delivery guarantees

- Ensures messages are delivered even if the receiver is temporarily unavailable (supports retries, persistence, etc.).

Scalability & load balancing

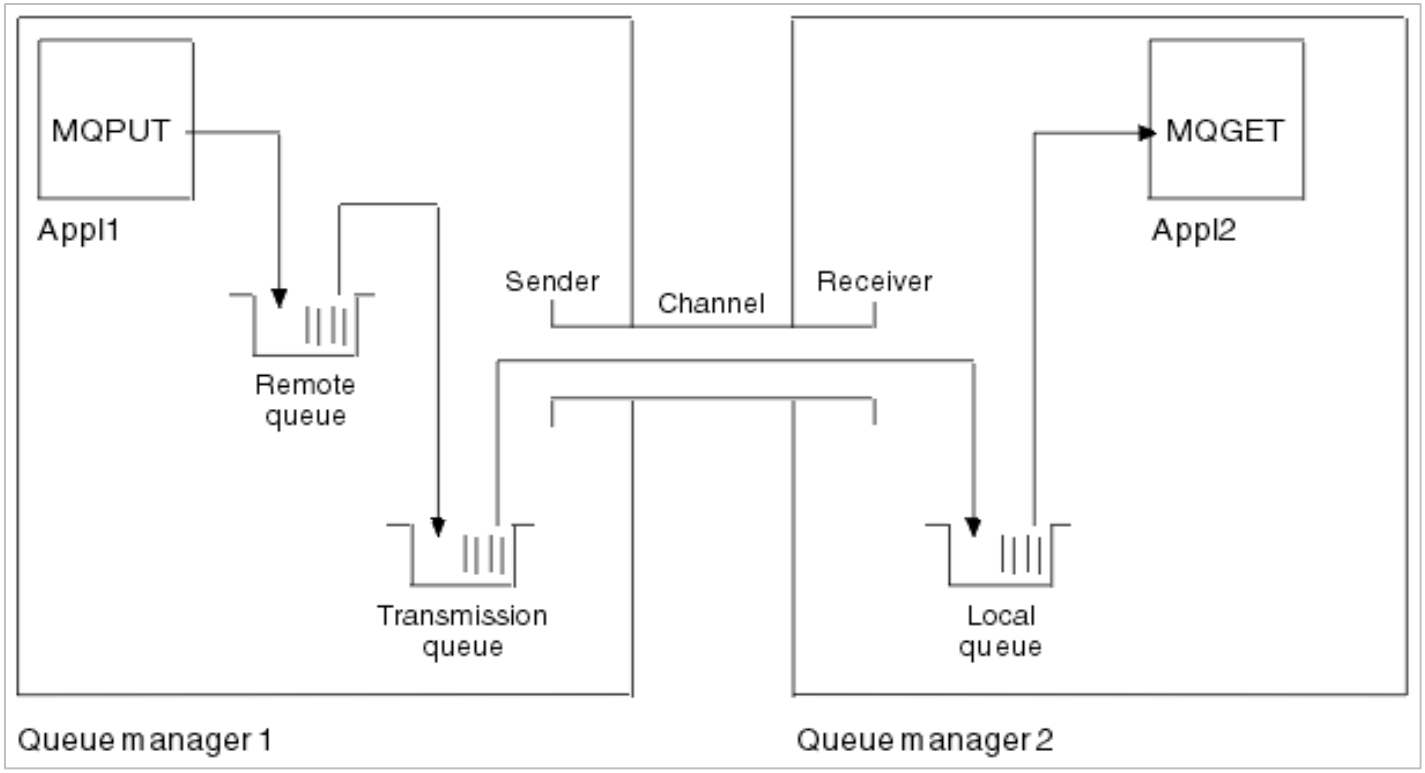
- Manages distribution of messages to multiple consumers for high throughput.

© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

Architectural Component

Message Queue



© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

Architectural Component

Message Queue

- Remote queues is to buffer the messages to be transmitted in case the Transmission queue couldn't catch up
- Transmission queues is to buffer the messages to be sent in case over the network the messages couldn't send as fast due to network latency
- Local queue is to buffer the messages received from sender in case App B isn't fast enough to receive and process

© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

Architectural Component

Examples of Message Brokers

- **RabbitMQ** – open-source, supports various messaging protocols
- **Apache Kafka** – high-throughput distributed event streaming platform
- **ActiveMQ** – flexible message broker by Apache
- **Amazon SQS** – cloud-based queue service
- **Redis Streams** – lightweight message broker functionality in Redis

© 2011-24 National University of Singapore. All Rights Reserved.

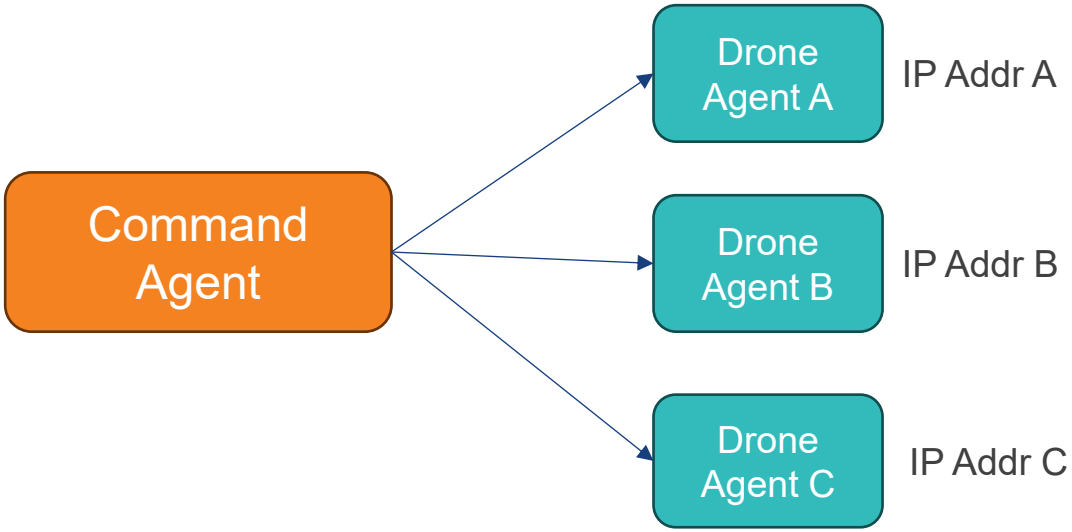
Architectural styles and component

Architectural Component

Why ?

Using the AI Rescue Simulation Case

Consider the scenario where the CommandAgent needs to communicate with multiple DroneAgents via REST API.

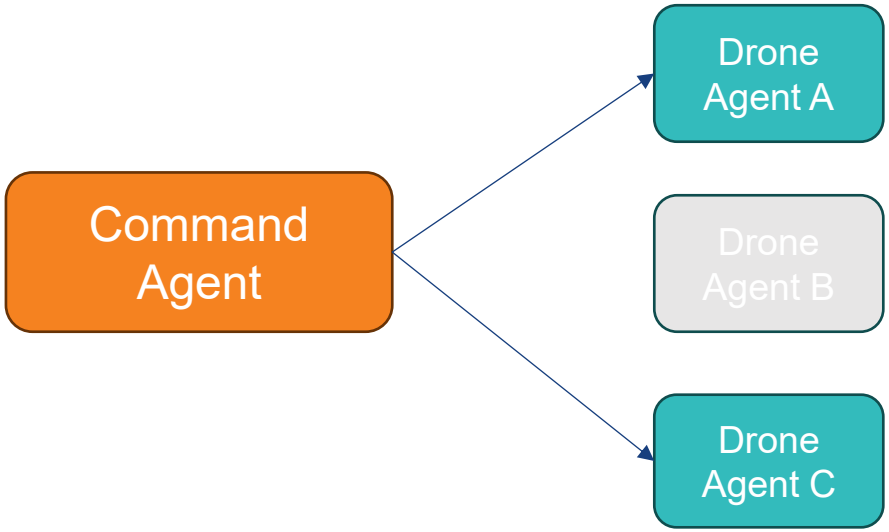


© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

Architectural Component

What happens if DroneAgent B for some reason is not work ?

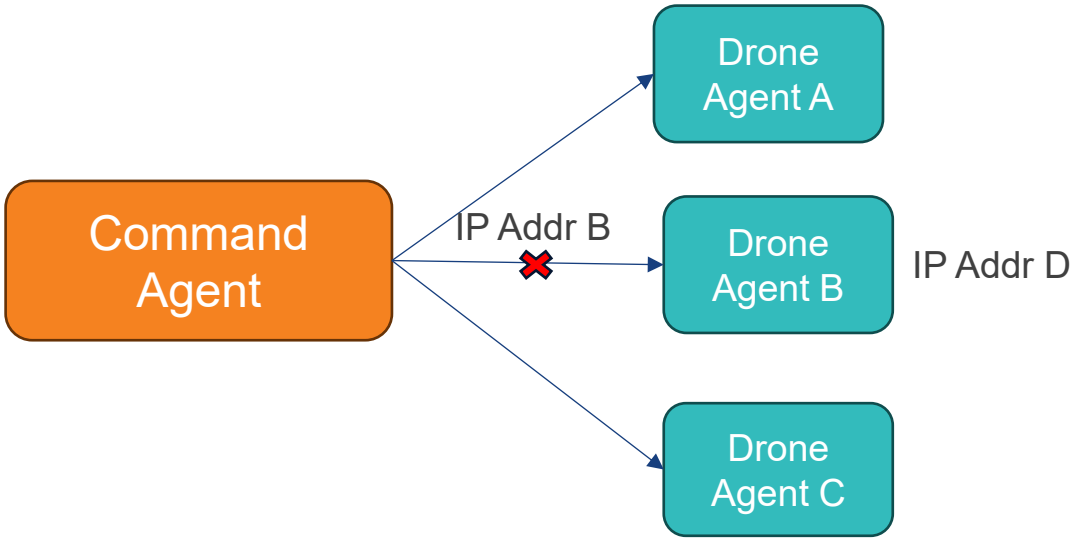


© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

Architectural Component

DroneAgent B is resurrected with a different IP address.



We can't hardcode the IP Address

© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

Architectural Component

Solution

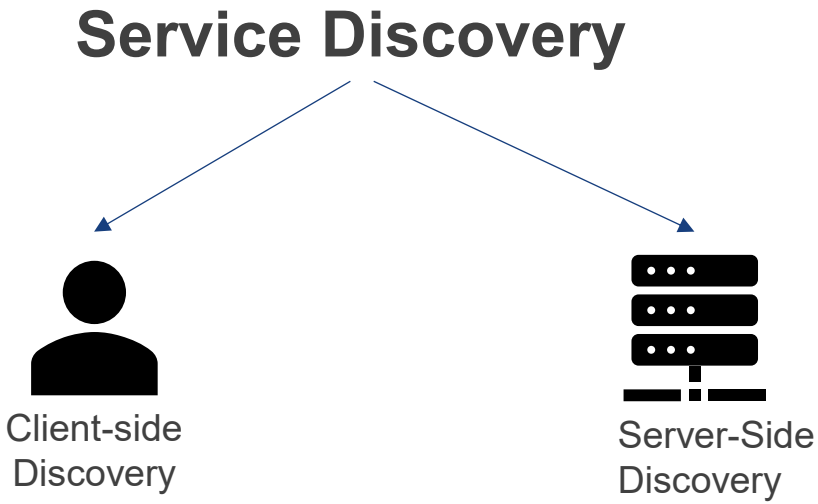
- A registry like DNS that stores the name of the service or agent and its corresponding IP address
- Caller will just need to call the name of the service or agent, and this component should lookup the registry and connect the it to the right address
- The registry should be updated should any services are created or destroyed

Service Discovery

© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

Architectural Component

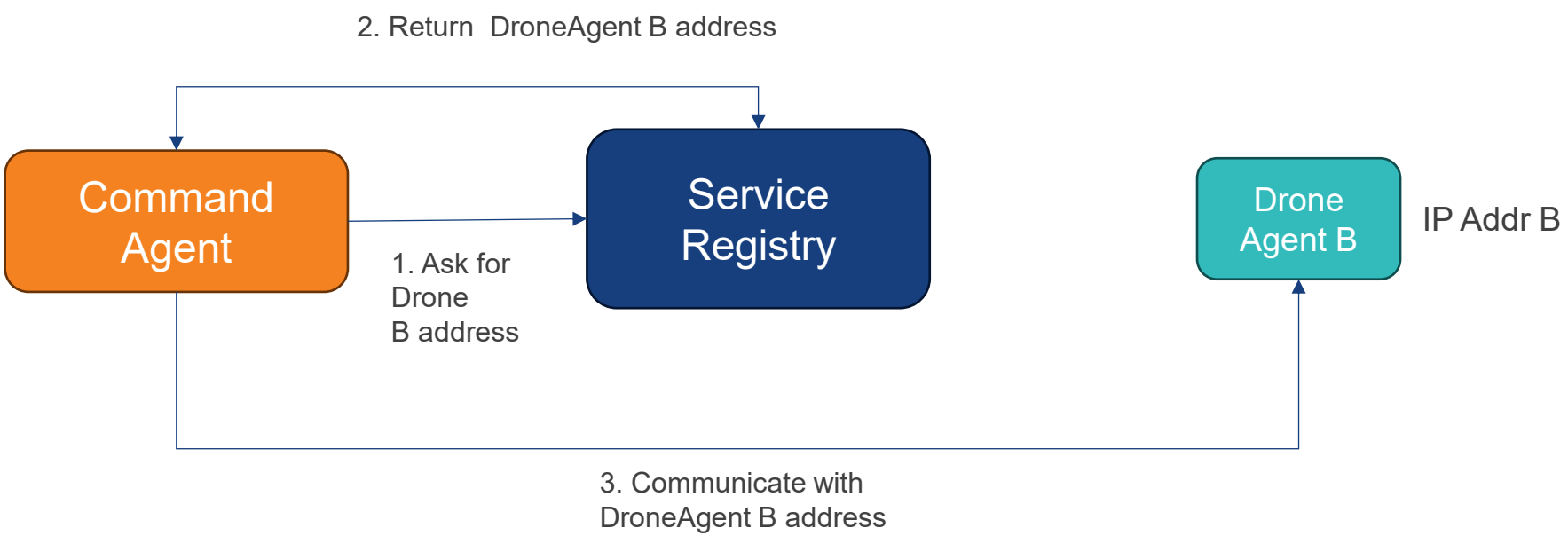


© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

Architectural Component

Service Discovery – Client-Side Discovery

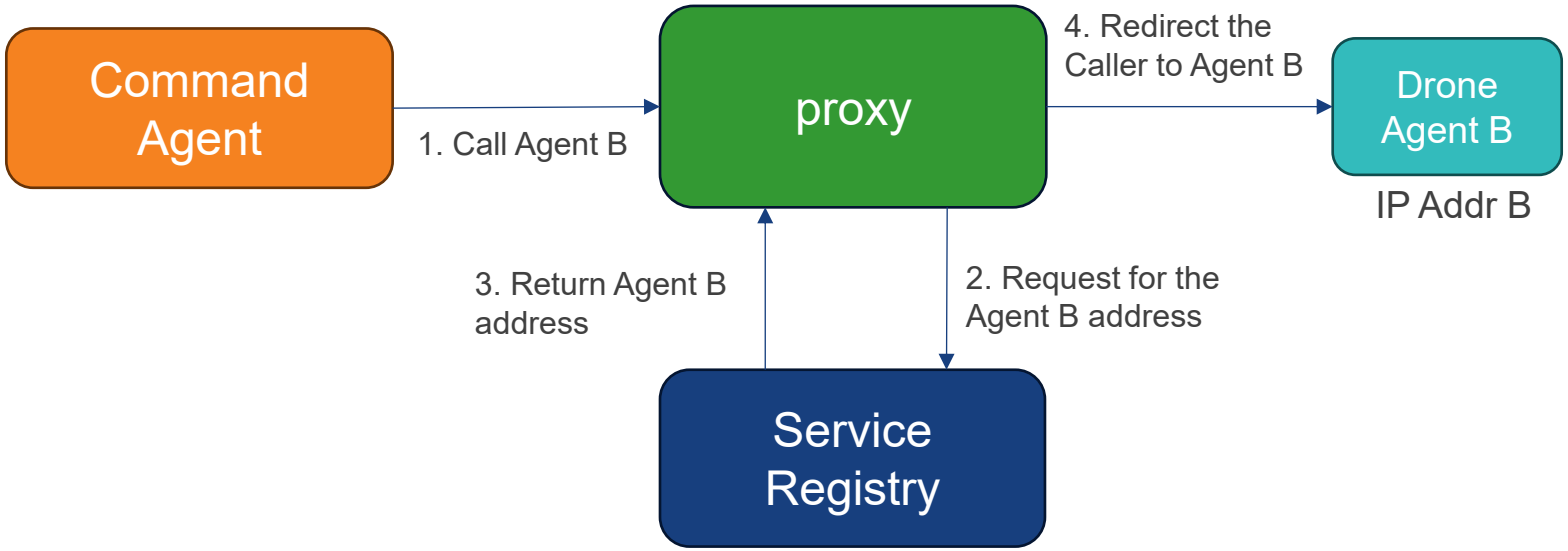


© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

Architectural Component

Service Discovery – Server-Side Discovery



Service Discovery is part of Service Mesh

© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

Architectural Component

Service Mesh

- Dedicated infrastructure layer that manages communication between services or agents in a distributed architecture
- Think of it as a smart network fabric that handles how services talk to each other reliably and securely

© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

Architectural Component

Service Mesh – Key Functions

- **Service Discovery** – knowing where services are running
- **Load Balancing** – distributing traffic among service instances
- **Traffic Management** – controlling routing and retries
- **Security** – encrypting service-to-service communication (mTLS)
- **Observability** – tracing, metrics, and logging for all traffic
- **Resilience** – timeouts, retries, rate limiting, circuit breakers

© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

Architectural Component

Service Mesh – Sidecar

- typically uses **sidecar proxies**
  - ❖ Small proxy processes run next to each service instance
  - ❖ Manage all incoming and outgoing traffic goes through the sidecar

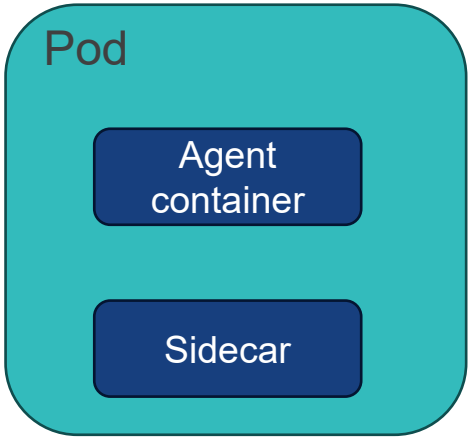
So what is Sidecar ?

© 2011-24 National University of Singapore. All Rights Reserved.

Architectural Component

Sidecar

- A **design pattern**
- A secondary container or process that runs **alongside** your main application or AI Agent in the same environment (e.g., same Kubernetes pod)



© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

Architectural Component

Sidecar – Common usage

- Service Mesh
  - Handling traffic routing, retries and security
- Logging/Monitoring
  - Collects logs/metrics and forwards them to monitoring systems
- Configuration Management
  - Fetches dynamic configuration
- Security
  - Manages TLS encryption/decryption
- Proxy/Gateway
  - Acts as reverse proxy to enforce policies or transform requests

© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

Architectural Component

Service Mesh – Istio

Istio is one of the most popular service meshes.

How it works:

- Uses **Envoy proxy** as a sidecar.

Provides:

- mTLS encryption between services.
- Fine-grained traffic routing.
- Telemetry (metrics, logs, traces).
- Policies (rate limiting, authentication).

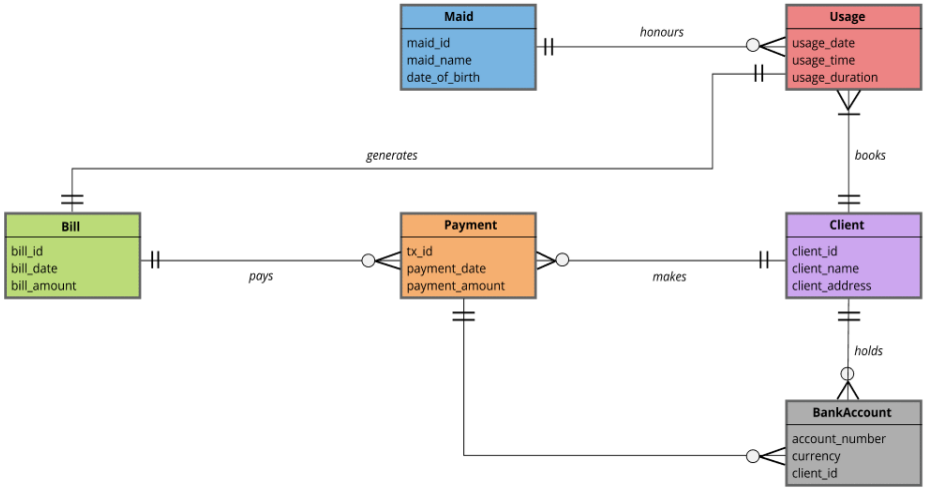
© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

Architectural Component

Database - SQL

- Relational database
- Data are organized in tabular format
- Each Domain Entity represents a table
- Each row of a table represents a record
- Each column of a table represents a data attribute (field)
- Using Structured Query Language (SQL) to perform data query, updates etc.



© 2011-24 National University of Singapore. All Rights Reserved.

Architectural Component

Database - SQL

Strengths

- **Structured & Organized Data**
  - ❖ Schemas are strongly enforced
- **Powerful Querying with SQL**
  - ❖ Could perform complex queries from different tables to generate insightful dashboard report
  - ❖ Efficient filtering and sorting
- **ACID Compliance**

  Ensures **reliable transactions** with:
  - ❖ Atomicity – all-or-nothing execution
  - ❖ Consistency – valid state maintained
  - ❖ Isolation – prevents conflicts in concurrent access
  - ❖ Durability – data persists after crashes

© 2011-24 National University of Singapore. All Rights Reserved.

Architectural Component

Database - SQL

Weakness

- **Rigid Structure**
  - ❖ Changing the schema (e.g., adding columns) can be complex and risky. Often it is also hard to rollback should the deployment fails
  - ❖ Not ideal for dynamic or unstructured data
- **Performance**
  - ❖ Could be slow because it needs frequent joining of many tables for transaction
- **Scalability**
  - ❖ Could only supports Vertical Scaling (ie adding more cpus, memories but not expanding for more instances)

© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

Architectural Component

Database - SQL

Usage in AI Agents

- **Fine grained access control**
  - ❖ Store which agents or users have access to what data/resources
  - ❖ SQL databases support role-based access control (RBAC) and auditing
- **Result Storage and Reporting**
  - ❖ Store results of agent computations, decisions, or simulations for review
  - ❖ Enables dashboards and analytics with SQL queries
- **AI Agents' State Data**
  - ❖ State data that are structured might be easier to be stored in SQL Database

© 2011-24 National University of Singapore. All Rights Reserved.

Architectural Component

Database – Vector Database

Because we have data like

Image

Videos

Audio

SM Posts

Unstructured Data

© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

Architectural Component

Database – Vector Database

Drone Agent
- *identify an animal who is trapped victim in a hill*
- *It identifies that the animal is a cat*

- How does the Drone Agent knows the animal is a cat ?
  - ➢ Tag various images of cats and stored in database ?
  - ➢ Uses pixel comparison ?

Tedious to implement

?

?

?

© 2011-24 National University of Singapore. All Rights Reserved.

Architectural Component

Database – Vector Database

- To solve the issues, we face in the previous slide
  - We need to convert those unstructured data into some form of representation to store
  - We also need fast retrieval and similarity search

Vector Database

© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

Architectural Component

Database – Vector Database

- Indexes and stores vector embeddings
- So that it will speed up retrieval and similarity search

Database in AI era

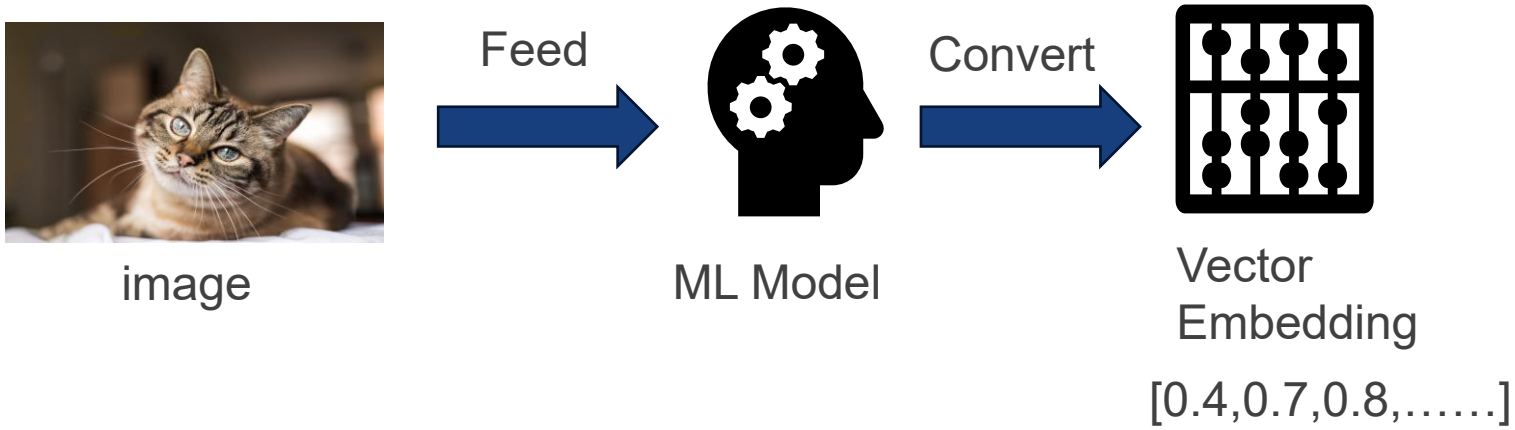
© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

Architectural Component

Database – Vector Database

How ?

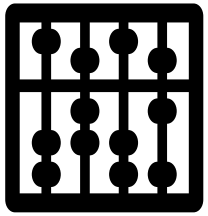


© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

Architectural Component

Database – Vector Database



Vector Embedding
[0.4,0.7,0.8,……]

- With the embeddings in place, the vector database is now able to find similar vectors
- Vector database find similar vectors by using nearest-neighbors search technique by computing metrics such as Cosine Similarity, Euclidean Distance etc
- But there is one problem it will be very computing intensive and slow for thousands of vectors

© 2011-24 National University of Singapore. All Rights Reserved.

Architectural Component

Database – Vector Database

- Index the vectors
- With indexing, the search based on distance metrics will be faster
- This can be a large topics and the details are omitted here

© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

Architectural Component

Database – Vector Database

Common Use Cases

- Multi-modal Search – eg like image search
- Text Similarity Search/Match
- Recommendation System
- Retrieval-Augmented Generation (RAG) to enhance the LLM response by referencing relevant documents

© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

Architectural Component

Database – Vector Database

Some of the popular Vector Databases

Vector DB	Language	Type	Highlights
Pinecone	Managed	Cloud	Fully managed, scalable, hybrid search, metadata filtering
Weaviate	Go	Open-source / Cloud	Built-in vectorizer, semantic + keyword search, GraphQL support
Chroma	Python	Local/lib	Simple, great for prototyping with LangChain or LlamaIndex
Redis (with RedisVector)	C	Open-source / Cloud	In-memory vector search with metadata + other data types
VESPA	Java	Open-source	Search engine with built-in vector support, good for ranking + hybrid

© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

Architectural Component

Monitoring Infra Monitoring –Infra Metrics

- CPU & memory usage
- Request volume – IOPS
- Application availability
- Peak usage time
- Average response time - Outliers and percentiles
- Request success & failure rate
- Request and Response payload size
- Garbage collection time

© 2011-24 National University of Singapore. All Rights Reserved.

Architectural styles and components

Architectural Component

Monitoring Infra Monitoring – Metric Tools Option

Prometheus

Time-series database that scrapes metrics from your application (usually via HTTP)

Grafana

Visualization tool that connects to Prometheus (or other data sources) to build dashboards

Any other Cloud platform tool like AWS CloudWatch, AWS X-Ray

© 2011-24 National University of Singapore. All Rights Reserved.

Architectural Component

Monitoring – LLM Monitoring Core Metrics

Metric	Purpose	Example Tools
Response Accuracy / Correctness	How factually correct or on-target the model's answers are	Human eval, GPT-4 judge, LangSmith evals
Hallucination Rate	% of outputs containing false info	Human-in-the-loop review, TruthfulQA
Token Usage	Cost tracking & prompt optimization	LangSmith

© 2011-24 National University of Singapore. All Rights Reserved.

Summary.

- The different architectural styles
- The different architectural components
  - Service Discovery
  - Different types of databases
  - Infrastructure Monitoring Metrics
  - Infrastructure Monitoring Tools
  - LLM Monitoring Metrics
  - LLM Monitoring Tools

© 2011-24 National University of Singapore. All Rights Reserved.